COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	8/5/26	**DATE SUBMITTED:**	8/7/26
NAME:	Condino, Niel Vincent B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5 **Queues**

1. PROCEDURE OUTPUT

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

//main driver
int main(){

    //create obj
    std::queue<char> bbq;

    //use the enqueue operation
    bbq.push('N');
    bbq.push('I');
    bbq.push('E');
    bbq.push('L');


    //call front
    std::cout<< "current front is: " << bbq.front() << std::endl;

    //call back
    std::cout << "current back is: " << bbq.back() << std::endl;

    //display
    display(bbq);
    std::cout<< "is orig list empty: " << bbq.empty() << std::endl;

    //use dequeue operation
    bbq.pop();
    display(bbq);
    bbq.pop();
    display(bbq);
    bbq.pop();
    display(bbq);
    bbq.pop();
    display(bbq);

```

```
40      std::cout<< "is orig list empty: " << bbq.empty() << std::endl;
41
42      return 0;
43  }
44
45  void display(std::queue<char> copyQ){
46      //create copy of queue
47      //Argument already creates a copy unless the parameter is uses reference, saves memory not creating another copy
48      //loop until empty
49      while (!copyQ.empty()){
50          //display the copy front
51          std::cout<< "current element:" << copyQ.front() << std::endl;
52          //pop the front
53          copyQ.pop();
54      }
55      //new line
56      std::cout << '\n';
57  }
```

Figure 1:ILOA main code

I saw that using STL's premade queue object makes implementation much faster. It familiarizes me with how the queue data structure works. The methods used by the object have similar names to the stack object from the STL. However, pop() works differently. Instead of popping the element of the queue, it pops the first element in the queue.

```
 cd "c:\Users\TIPQC\OneDri
current front is: N
current back is: L
current element:N
current element:I
current element:E
current element:L

is orig list empty: 0
current element:I
current element:E
current element:L

current element:E
current element:L

current element:L


is orig list empty: 1
```

Figure 2: ILOA output

The figure above shows what the code outputs. First, it shows the front and back of the list. Even though I did not create a copy of the queue explicitly in the function, popping inside it does not affect the original list. This is shown by the line "is orig list empty:". After this, I popped all the elements of the queue in the main function. This is shown by the queue elements going down in size

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

//node class
template<typename T>
class qNode{
    public:
    T data; //contains data to be stored in the node
    qNode* next = nullptr;
};

//new node creator
template <typename T>
qNode<T>* createNewNode(T newData){
    qNode<T> *newNode = new qNode<T>;

    newNode->data = newData;
    newNode->next = nullptr;

    return newNode;
}

//isempty
template<typename T>
bool isQEmpty(qNode<T>*frontPtr,qNode<T>*backPtr){
    if (frontPtr == nullptr && backPtr == nullptr){
        return true;
    }
    return false;
}

//enqueue
//double pointer to modify the pointers
template<typename T>
void enqueue(T newData,qNode<T>** frontPtr,qNode<T>** backPtr){
    //create new node
    qNode<T>* newNode = createNewNode(newData);
    //check if queue empty
    if (isQEmpty(*frontPtr,*backPtr)){
```

```
40          (*frontPtr) = newNode;
41          (*backPtr) = newNode;
42      }
43      //if not insert at back
44      else{
45          (*backPtr)->next = newNode;
46          (*backPtr) = newNode;
47      }
48  }
49
50  //dequeue
51  template<typename T>
52  void dequeue(qNode<T>** frontPtr,qNode<T>** backPtr){
53      //check if q empty
54      if (isQEmpty(*frontPtr,*backPtr)){
55          std::cout << "queue empty"<<std::endl;
56          return;
57      }
58      //create temp var to store node to be delted
59      qNode<T>* toBeDeleted = nullptr;
60      //assign the current front to the toBeDeleted
61      toBeDeleted = (*frontPtr);
62      //check if the q has 1 item
63      if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
64          //only one item in queue
65          *frontPtr = nullptr;
66          *backPtr = nullptr;
67          delete toBeDeleted;
68          return;
69      }
70      //shift the front node
71      (*frontPtr) = (*frontPtr)->next;
72      //unlink the node to be deleted
73      toBeDeleted->next = nullptr;
74      //delete the node
75      delete toBeDeleted;
```

```
77
78  //display
79  template<typename T>
80  void displayQ(qNode<T>* front){
81      if (front == nullptr){
82          std::cout << "There is no list\n";
83      }
84      while(front!= nullptr){
85          std::cout<< front->data << " \n";
86          front = front->next;
87      };
88  }
89
90  #endif
```

Figure 3: Queue Linked List Implementation Header File

I find this approach much more compatible with the format of a linked list. This structure keeps track of the front and the tail of the linked list. Because the elements are just linked together, it is easy to insert and remove elements from this queue using the linked list approach. To keep track of the front and back of the queue, pointers are used to identify them. Most of the operations in this module closely resemble the operations from the original linked list module. There are small alterations to accommodate the queue data structure. The last function, displayQ, just loops through the list and prints the data. This is just a repurposing of the link traversal function from the previous module.

```cpp
#include <iostream>
#include "queueLL.h"

//main driver
int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout << "Testing enqueue:\n";
    enqueue('N',&front,&back);
    std::cout <<"front: " <<front->data << " back: " << back->data << '\n';
    enqueue('I',&front,&back);
    std::cout <<"front: " <<front->data << " back: " << back->data << '\n';
    enqueue('E',&front,&back);
    std::cout <<"front: " <<front->data << " back: " << back->data << '\n';
    enqueue('L',&front,&back);
    std::cout <<"front: " <<front->data << " back: " << back->data << '\n';

    std::cout << "Display queue: \n";
    displayQ(front);
    std::cout << "Testing dequeue: \n";
    dequeue(&front,&back);
    std::cout <<"front: " <<front->data << " back: " << back->data << '\n';
    dequeue(&front,&back);
    std::cout <<"front: " <<front->data << " back: " << back->data << '\n';
    dequeue(&front,&back);
    std::cout <<"front: " <<front->data << " back: " << back->data << '\n';
    dequeue(&front,&back);
    //List is empty from the line above
    dequeue(&front,&back);
    std::cout << "Is q empty: " << isQEmpty(front,back);
    //std::cout <<"front: " <<front->data << " back: " << back->data << '\n';
    return 0;
}
```

Figure 4: ILO B1 main code

The code above is used to test the code from the previous figure. For the first few lines of the code, it enqueues the letters N, I, E, and L. It then displays the data inside the queue using the displayQ() function. One by one, it dequeues all the elements in the list. Lastly, shows a message confirming if the queue is empty after all the dequeuing.

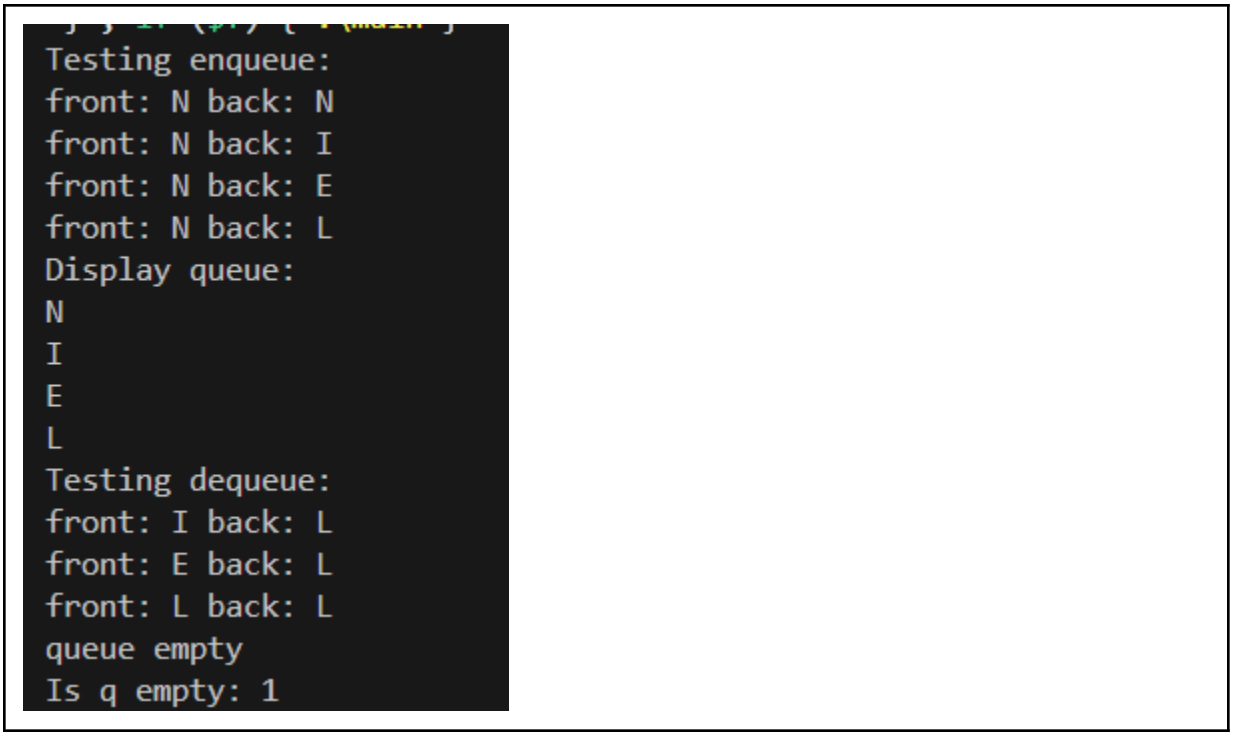


Figure 5: ILO B1 main code output

The output confirms what is happening with the code. Enqueuing other data to the list does not modify the top or front of the queue. Instead, it appends the new element to the tail of the list. displayQ() then prints out the elements in the order of their insertion, respecting the First in, First out approach. Dequeuing then modifies the front of the list, removing these elements from the queue. This is done one by one until the list is empty. The last line confirms there are no elements in the list.

```cpp
template<typename T>
class QueueArray{
    private:
        //pointer to dynamically allocate array
        T* q_arr;
        //maximum num of elem
        std::size_t q_capacity;
        //current number of elements
        std::size_t q_size;
        //Index of front
        int q_front;
        //index of back
        int q_back;
    public:
    //Constructor
    QueueArray(std::size_t capacity = 10);
    //Destructor
    ~QueueArray();
    //Copy const
    QueueArray(const QueueArray& other);
    //Copy assignment operator
    QueueArray& operator=(const QueueArray& other);
    //q operations
    bool Full();
    bool Empty();
    size_t Size();
    void Clear();
    T Front();
    T Back();
    void Enqueue(T value);
    T Dequeue();
};

//Constrictor & Destructor
template<typename T>
QueueArray<T>::QueueArray(std::size_t capacity){
    q_arr = new T[capacity];
    q_capacity = capacity;
    //default size,front,and back
    Clear();
}

```

```cpp
template<typename T>
QueueArray<T>::~QueueArray(){
    delete[] q_arr;
}

template<typename T>
QueueArray<T>::QueueArray(const QueueArray &other){
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    // Allocate new array memory
    q_arr = new T[q_capacity];

    // Deep copy existing elements
    for (std::size_t i = 0; i < q_capacity; i++) {
        q_arr[i] = other.q_arr[i];
    }
}

template<typename T>
QueueArray<T>& QueueArray<T>::operator=(const QueueArray &other){
    //Prevent copying self
    if (this == &other){
        return *this;
    }

    //Delete old pointer to prevent memory leaks
    delete[] q_arr;

    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    // Allocate new array memory
    q_arr = new T[q_capacity];

    // Deep copy existing elements
    for (std::size_t i = 0; i < q_capacity; i++) {
        q_arr[i] = other.q_arr[i];
    }

    return *this;
}
```

```cpp

//Operations
template <typename T>
bool QueueArray<T>::Full(){
    return q_size == q_capacity;
}

template <typename T>
bool QueueArray<T>::Empty(){
    return q_size == 0;
}

template <typename T>
size_t QueueArray<T>::Size() {
    // RETURN CURRENT SIZE OF QUEUE
    return q_size;
}

template <typename T>
T QueueArray<T>::Front() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T();
    }
    // Return the front
    return q_arr[q_front];
}

template <typename T>
T QueueArray<T>::Back() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T();
    }
    // Return the back
    return q_arr[q_back];
}

// Clear()
template <typename T>
void QueueArray<T>::Clear() {
    // Reset the queue to its initial state
    q_size = 0;
    q_front = 0;
    q_back = -1; // Set back to -1 since there are no elements
}
```

```cpp
// Enqueue()
template <typename T>
void QueueArray<T>::Enqueue(T value) {
    // Check if the queue is full
    if (Full()) {
        std::cout << "Queue is Full\n";
        return;
    }
    // Increment the back
    q_back = (q_back + 1) % q_capacity;
    // Add the new value to the back of the queue
    q_arr[q_back] = value;
    // Increment the size of the queue
    q_size++;
}

// Dequeue()
template <typename T>
T QueueArray<T>::Dequeue() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T();
    }
    // Store the front value
    T value = q_arr[q_front];
    // Increment the front index in a circular manner
    q_front = (q_front + 1) % q_capacity;
    // Decrement the size of the queue
    q_size--;
    //reset index if empty
    if (Empty()){
        Clear();
    }
    // Return the dequeued value
    return value;
}

#endif
```

Figure 6: ILO B2 Array-based queue implementation

The code above implements a queue data structure on static arrays. It has different methods for “observation” and “modification” of the queue. The observation functions, such as front () and back(), do not modify the queue; instead, they return a value from the queue. The modifier functions, such as enqueue() and dequeue(), change the elements inside the list. To implement a circular approach, the modulo operator is used for determining the index of the current front. Instead of shifting the elements, the index of the front and back is shifted. To discuss the big 3 for classes, the destructor deletes the array allocated in memory to prevent memory leaks. The copy assignment operator and constructor then create a deep copy of the object. They are mostly the same with their copying, with only added checks for the copy assignment operator.



```cpp
#include <iostream>
#include "queueArr.h"

//main driver
int main(){
    QueueArray<int> q_Arr (capacity: 5);
    std::cout << "Queue size is " << q_Arr.Size() << '\n';

    std::cout << "Current front: " << q_Arr.Front() << '\n';
    std::cout << "Current back: " << q_Arr.Back() << '\n';

    //put elements in queue
    for (int i = 1; i <= 6;i++){
        std::cout << "Tried to Enqued: " << i << '\n';
        q_Arr.Enqueue(value: i);
        std::cout << "Current front: " << q_Arr.Front() << '\n';
        std::cout << "Current back: " << q_Arr.Back() << '\n';
    }
    std::cout << "Queue size is " << q_Arr.Size() << '\n';

    std::cout << "Current front: " << q_Arr.Front() << '\n';
    std::cout << "Current back: " << q_Arr.Back() << '\n';

    std::cout << "Dequeing: " << q_Arr.Dequeue() << '\n';

    std::cout << "Current front: " << q_Arr.Front() << '\n';
    std::cout << "Current back: " << q_Arr.Back() << '\n';

    //Clear array
    q_Arr.Clear();

    std::cout << "Is array empty: " << (q_Arr.Empty()? "yes": "no") << '\n' ;

    return 0;
}
```

Figure 7: ILO B2 main code

To test out the QueueArray class, this driver code is created. It initializes an object from the QueueArray class. It prints out the size and the current elements at the front and back of the queue. Using a loop, integers are queued in the array. The size of the queue is then checked again, along with the front and back elements of the queue. The dequeue is then tested to check if it removes the front of the queue. After this, the front and back of the queue are then inspected. The clear function, which just resets the “pointers” for the front and back of the queue instead of removing all the elements in the array, is then tested. The array is then checked if it is “empty” at the end of the code.

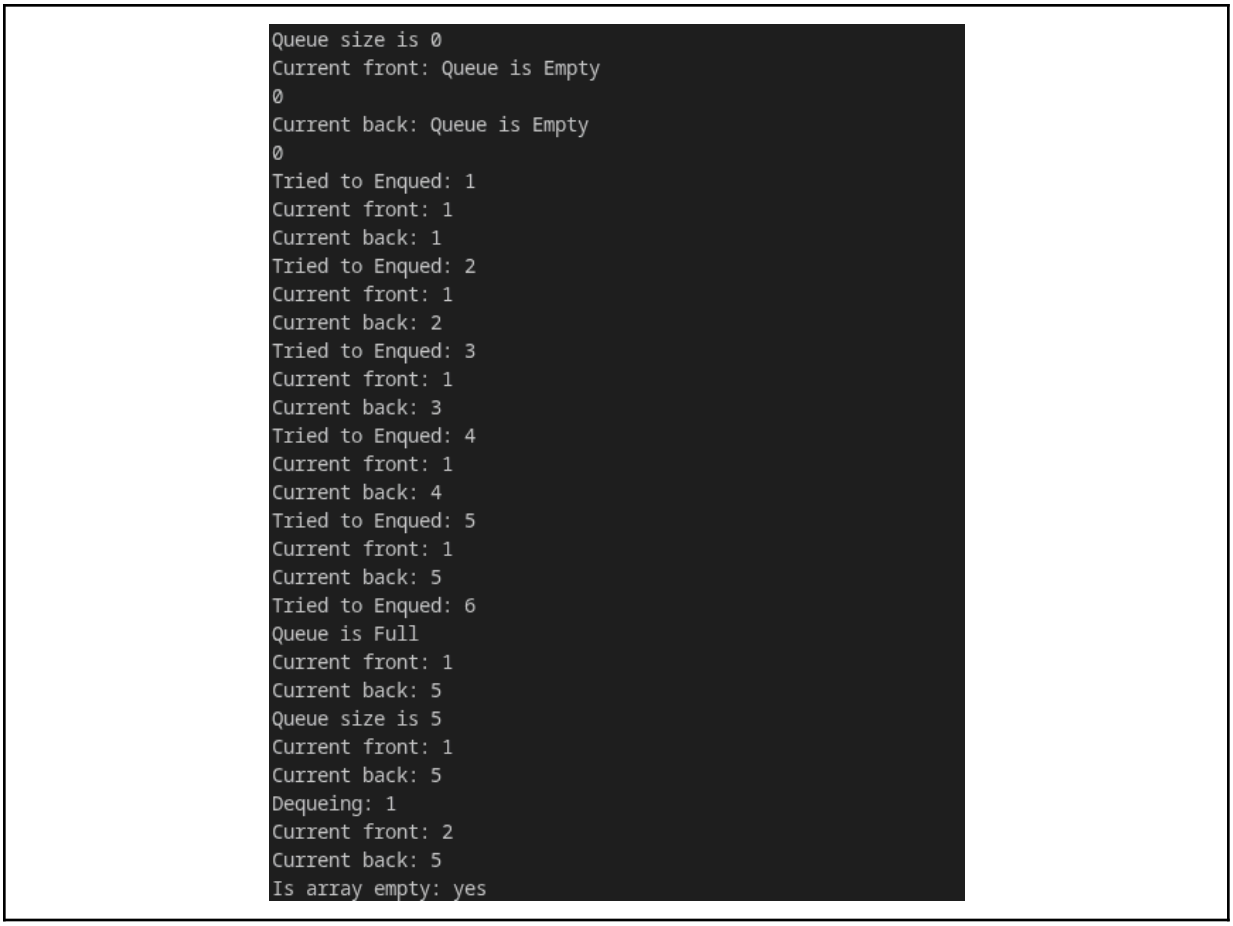


Figure 8: ILO B2 main code output

The output above shows that at first, the queue's size is 0, signifying that the queue is empty. A series of enqueuing then happened. The 6th element tried but was not let into the queue due to the array having a maximum capacity. Dequeuing 1 makes 2 the current front while the back stays as is. Lastly, the array is "cleared" after calling the Clear() method, confirmed by the last line.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#ifndef PRINTERJOB_H
#define PRINTERJOB_H

#include <iostream>
#include "../ILOB/queueLL.h"

class Job{
    private:
        std::string name;
        int pages;
    public:
        int ID;

        Job(std::string nameOfJob= " ", int jobPages=0){
            name = nameOfJob;
            pages = jobPages;
        }

        std::string getName(){
            return name;
        }

        int getPages(){
            return pages;
        }
};

class Printer{
    private:
        qNode<Job>* front = nullptr;
        qNode<Job>* back = nullptr;

        int trackingID = 0;
        int jobLeft = 0;

    public:
    Printer(){
        std::cout << "================================= Printer Initialized =================================\n";
    }
    //remove remaining data in queue if it is not dequeued
    ~Printer(){
        std::cout << "================================= Printer Closed ======================================\n";
        while (!isQEmpty(frontPtr: front,backPtr: back)){
            dequeue(frontPtr: &front,backPtr: &back);
        }
    }

```

```
48      //Add Job
49      void queueJob(Job sentJob){
50          sentJob.ID = ++trackingID;
51          enqueue(newData: sentJob, frontPtr: &front, backPtr: &back);
52          jobLeft++;
53          std::cout << " ====================== Queued Job ================================\n";
54      }
55
56      //Do Job
57      void performJob(){
58          //check if there are jobs left
59          if (jobLeft == 0) {
60              std::cout << "No more jobs left\n";
61              return;
62          }
63
64          Job jobToBePerformed = dequeue(frontPtr: &front, backPtr: &back);
65          jobLeft--;
66          std::cout << "============== PRINTING ============================\n";
67          std::cout << "Printed Job:\n";
68          std::cout << "ID: " << jobToBePerformed.ID << '\n';
69          std::cout << "Job sent by: " << jobToBePerformed.getName() << '\n';
70          std::cout << "Job Pages: \n" << jobToBePerformed.getPages() << '\n';
71          std::cout << "----------------------------------------------------\n";
72      }
73
74      void performAllJob(){
75          while (!isQEmpty(frontPtr: front, backPtr: back)){
76              performJob();
77          }
78      }
79      void showJobQueue(){
80          std::cout << "Jobs left: " << jobLeft << '\n';
81      }
82  };
83
84  #endif
```

Figure 9: Print Queue Code

The printer class makes use of the linked list implementation for its queue system. This implementation is used because a theoretically large number of jobs can be queued as long as the memory limit is not reached. While this is not CPU-cache-friendly and each element has larger memory allocations, I imagine this system is not part of the actual printer but a server that decides what the printer will print. This class acts as an interface from the program where the print job will come from to itself, the server, and the printer. A corporate environment demands a lot of paperwork, and adding a hard cap to the printer can cause delays in this aspect, which can cost money. Opting for this solution solves the problem but has its own trade-offs as mentioned above.

```cpp
#include <iostream>
#include "Printer&Job.h"

//main driver
int main(){
    Printer myPrinter;

    //simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one
    for (int i = 1; i <= 15; i++){
        std::cout << "Person " << i << " sent a job\n";
        std::string jobName = "Job " + std::to_string(val: i);
        Job newJob(nameOfJob: jobName,jobPages: i);
        myPrinter.queueJob(sentJob: newJob);
    }

    myPrinter.showJobQueue();

    myPrinter.performJob();

    myPrinter.showJobQueue();

    myPrinter.performAllJob();

    myPrinter.showJobQueue();

    myPrinter.performJob();

    myPrinter.showJobQueue();

    return 0;
}
```

Figure 10: Driver code

This code simulates multiple people sending a job to the printer. A for loop was used to simulate this action. It sends out a print job from 15 different people. These jobs are then put inside the queue. It then shows how many jobs are in the queue. It then uses the performJob() method to “print” out the first print job. The number of jobs is then requested again. It then asks the printer to print all the pending print jobs in the queue. It then tries to show the remaining jobs left in the queue. It then tries to perform A job even when there is nothing in the queue. Lastly, it shows the queue is empty. The printer is then closed, and the simulation is terminated.

```
================================== Printer Initialized ======================
Person 1 sent a job
 ====================== Queued Job ================================
Person 2 sent a job
 ====================== Queued Job ================================
Person 3 sent a job
 ====================== Queued Job ================================
Person 4 sent a job
 ====================== Queued Job ================================
Person 5 sent a job
 ====================== Queued Job ================================
Person 6 sent a job
 ====================== Queued Job ================================
Person 7 sent a job
 ====================== Queued Job ================================
Person 8 sent a job
 ====================== Queued Job ================================
Person 9 sent a job
 ====================== Queued Job ================================
Person 10 sent a job
 ====================== Queued Job ================================
Person 11 sent a job
 ====================== Queued Job ================================
Person 12 sent a job
 ====================== Queued Job ================================
Person 13 sent a job
 ====================== Queued Job ================================
Person 14 sent a job
 ====================== Queued Job ================================
Person 15 sent a job
 ====================== Queued Job ================================
Jobs left: 15
```

```
============== PRINTING ============================
Printed Job:
ID: 1
Job sent by: Job 1
Job Pages:
1
--------------------------------------------------
Jobs left: 14
============== PRINTING ============================
Printed Job:
ID: 2
Job sent by: Job 2
Job Pages:
2
--------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 3
Job sent by: Job 3
Job Pages:
3
--------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 4
Job sent by: Job 4
Job Pages:
4
--------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 5
Job sent by: Job 5
Job Pages:
5
--------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 6
Job sent by: Job 6
Job Pages:
6
--------------------------------------------------
```

```
============== PRINTING ============================
Printed Job:
ID: 7
Job sent by: Job 7
Job Pages:
7
--------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 8
Job sent by: Job 8
Job Pages:
8
--------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 9
Job sent by: Job 9
Job Pages:
9
--------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 10
Job sent by: Job 10
Job Pages:
10
--------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 11
Job sent by: Job 11
Job Pages:
11
--------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 12
Job sent by: Job 12
Job Pages:
12
```

```
============== PRINTING ============================
Printed Job:
ID: 13
Job sent by: Job 13
Job Pages:
13
-----------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 14
Job sent by: Job 14
Job Pages:
14
-----------------------------------------------------
============== PRINTING ============================
Printed Job:
ID: 15
Job sent by: Job 15
Job Pages:
15
-----------------------------------------------------
Jobs left: 0
No more jobs left
Jobs left: 0
=================================== Printer Closed ========================================
```

Figure 11: Output of driver code

The output above shows what the printer object from the last figure does. It first shows the enqueuing of jobs from multiple people. These jobs are stored inside the queue. Around 15 jobs were inserted into the queue. Each job is then printed using the performJob() and performAllJobs() methods as shown by the output. The printing of documents started from Person number 1 to 15, obeying the First in First Out principle of queues. Trying to call the same method just returns an error message when there are no print jobs in the queue. Lastly, the printer's deconstructor is called.

Conclusion:

In this activity, I was able to learn more about how the concept of queues connects to programming. I learned that this data structure applies the First in First Out principle, which is the opposite of the stack data structure. Testing it out first on a premade object from the STL library made it easier to familiarize. I learned that this data structure can also be implemented in an array or a linked list, the same as the previously discussed structure. I was able to see how easy it is to implement on a linked list, while implementing in an array takes some complexity. To have the same time complexity as a linked list implementation (O(1)), a circular queue is implemented. Instead of shifting the elements, the “front” and “back” indices are the ones that are shifted. The supplementary part’s decision stems from the time complexity and capacity of using a linked list. Throughout the activity, I found that I need to improve my thinking of finding optimal ways of implementing algorithms in code. Overall, this activity showed how to implement a queue in different ways. I intend to use this newfound concept to solve problems in programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.